Лабораторна робота №4

Поведінкові шаблони

Мета роботи: навчитися реалізовувати структурні шаблони проєктування Ланцюжок відповідальностей, Посередник, Спостерігач, Стратегія, Мементо

Термін виконання: 2 год (1 заняття)

Зміст роботи

Завдання №1

1. Створіть систему підтримки користувачів.

2. Вона має функціонувати так, як покрокове меню (наприклад, коли телефонуєте до оператора мобільного зв'язку). Але замість голосових повідомлень Ви маєте виводити відповідні повідомлення в консоль і чекати на вибір користувача.

3. Система повинна мати мінімум 4 рівні. Всі питання мають на меті обрати правильний рівень підтримки (тобто Handler) для користувача. Тобто вже на першому питанні може бути підібрано правильний рівень підтримки, тоді меню закінчується. А може бути так, що на жодному питанні не буде знайдено правильний рівень (Handler), тоді меню має повторитися.

4. Покажіть правильність роботи свого коду запустивши його в головному методі програми.

Лістинг програми:

ExpertSupport:

```
class ExpertSupport : SupportHandler
{
    public override void HandleRequest(int level)
    {
        if (level == 4)
        {
            Console.WriteLine("Вас обслуговує Експертна підтримка.");
        }
        else
        {
            Console.WriteLine("Не вдалося знайти відповідний рівень підтримки.");
        }
    }
     }
```

AdminSupport:

```
class AdminSupport : SupportHandler
{
    public override void HandleRequest(int level)
    {
        if (level == 3)
        {
            Console.WriteLine("Вас обслуговує Адміністратор.");
        }
        else if (next != null)
        {
            next.HandleRequest(level);
        }
    }
     }
```

TechnicalSupport:

```
class TechnicalSupport : SupportHandler
{
    public override void HandleRequest(int level)
    {
        if (level == 2)
        {
            Console.WriteLine("Вас обслуговує Технічна підтримка.");
        }
        else if (next != null)
        {
            next.HandleRequest(level);
        }
    }
     }
```

BasicSupport:

```
class BasicSupport : SupportHandler
{
    public override void HandleRequest(int level)
    {
        if (level == 1)
        {
            Console.WriteLine("Вас обслуговує Базова підтримка.");
        }
        else if (next != null)
        {
            next.HandleRequest(level);
        }
    }
     }
```

SupportHandler:

```
abstract class SupportHandler : ISupportHandler
{
    protected ISupportHandler next;

    public void SetNext(ISupportHandler nextHandler)
    {
        next = nextHandler;
    }

    public abstract void HandleRequest(int level);
     }
```

ISupportHandler:

```
interface ISupportHandler
```

```
{
    void SetNext(ISupportHandler next);
    void HandleRequest(int level);
     }
```

Program:

```
using System.Text;

class Program
{
    static void Main()
    {
        Console.InputEncoding = Encoding.UTF8;
        Console.OutputEncoding = Encoding.UTF8;
        BasicSupport basic = new BasicSupport();
        TechnicalSupport tech = new TechnicalSupport();
        AdminSupport admin = new AdminSupport();
        ExpertSupport expert = new ExpertSupport();

        basic.SetNext(tech);
        tech.SetNext(admin);
        admin.SetNext(expert);

        while (true)
        {
            Console.WriteLine("=== Система підтримки ===");
            Console.WriteLine("1 - Проблеми з паролем");
            Console.WriteLine("2 - Проблеми з програмою");
            Console.WriteLine("3 - Проблеми з доступом");
            Console.WriteLine("4 - Складна технічна проблема");
            Console.WriteLine("0 - Вихід");

            Console.Write("Ваш вибір: ");
            int choice = int.Parse(Console.ReadLine());

            if (choice == 0)
                break;

            basic.HandleRequest(choice);

            Console.WriteLine();
        }
    }
     }
```

Результат виконання програми:

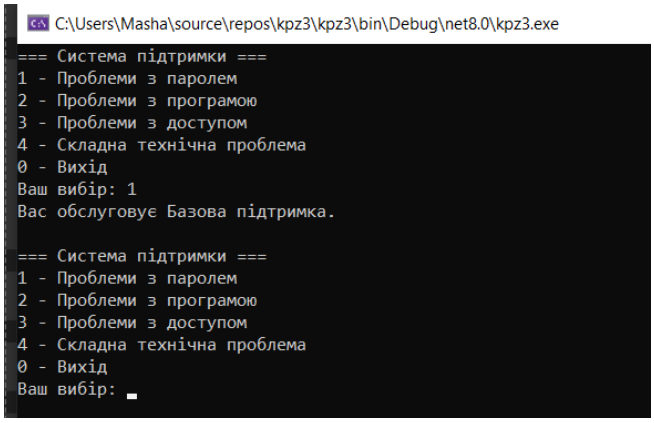

Рис1. Результат виконання програми.

Завдання 2.

1. Відрефакторте код продемонстрований на лекції за допомогою використання шаблону Посередник.

2. В результаті рефакторингу Aircraft не повинен “знати” про Runway і навпаки. Обидві сутності повинні “знати” лише про CommandCentre.

3. Покажіть правильність роботи свого коду запустивши його в головному методі програми.

Лістинг програми:

CommandCentre:

```
namespace DesignPatterns.Mediator
{
    class CommandCentre
    {
        private List<Runway> runways = new();
        private Dictionary<Aircraft, Runway> aircraftRunway = new();

        public CommandCentre(List<Runway> runways)
        {
            this.runways = runways;
        }

        public void RequestLanding(Aircraft aircraft)
        {
            Console.WriteLine("Checking runways for landing...");

            foreach (var runway in runways)
            {
                if (!runway.IsBusy)
                {
                    Console.WriteLine($"Aircraft {aircraft.Name} landed on runway
{runway.Id}");
                    runway.IsBusy = true;
                    runway.HighLightRed();

                    aircraftRunway[aircraft] = runway;
                    return;
                }
            }

            Console.WriteLine("No free runway available.");
        }

        public void RequestTakeOff(Aircraft aircraft)
        {
            if (aircraftRunway.ContainsKey(aircraft))
            {
                var runway = aircraftRunway[aircraft];

                Console.WriteLine($"Aircraft {aircraft.Name} is taking off from runway
{runway.Id}");

                runway.IsBusy = false;
                runway.HighLightGreen();
```

```
                aircraftRunway.Remove(aircraft);
            }
            else
            {
                Console.WriteLine($"Aircraft {aircraft.Name} is not on any runway.");
            }
        }
    }
     }
```

Runway:

```
namespace DesignPatterns.Mediator
{
    class Runway
    {
        public readonly Guid Id = Guid.NewGuid();
        public bool IsBusy;

        public void HighLightRed()
        {
            Console.WriteLine($"Runway {Id} is busy!");
        }

        public void HighLightGreen()
        {
            Console.WriteLine($"Runway {Id} is free!");
        }
    }
     }
```

Aircraft:

```
namespace DesignPatterns.Mediator
{
    class Aircraft
    {
        public string Name;
        public bool IsTakingOff { get; set; }

        private CommandCentre mediator;

        public Aircraft(string name, CommandCentre mediator)
        {
            Name = name;
            this.mediator = mediator;
        }

        public void Land()
        {
            Console.WriteLine($"Aircraft {Name} requests landing.");
            mediator.RequestLanding(this);
        }

        public void TakeOff()
        {
            Console.WriteLine($"Aircraft {Name} requests takeoff.");
            mediator.RequestTakeOff(this);
        }
    }
     }
```

Program:

```
using DesignPatterns.Mediator;
```

```
class Program
{
    static void Main()
    {
        Runway runway1 = new Runway();
        Runway runway2 = new Runway();

        var runways = new List<Runway> { runway1, runway2 };

        CommandCentre commandCentre = new CommandCentre(runways);

        Aircraft aircraft1 = new Aircraft("Boeing 737", commandCentre);
        Aircraft aircraft2 = new Aircraft("Airbus A320", commandCentre);

        aircraft1.Land();
        aircraft2.Land();

        aircraft1.TakeOff();
        aircraft2.TakeOff();
    }
 }
```

Результат виконання програми:

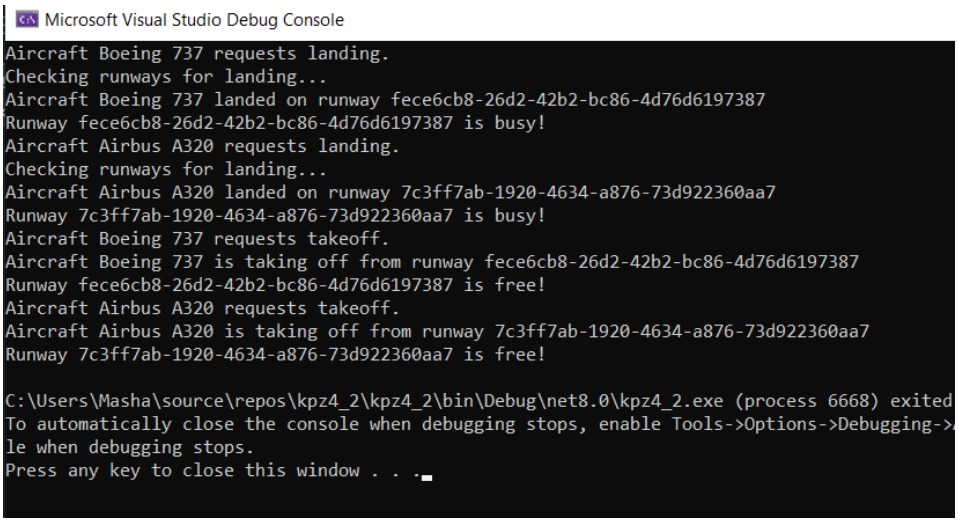

Рис2. Результат виконання програми.

Завдання 3.

1. Додайте до Вашого LightHTML з завдання 5 ЛР №3 можливість додавання EventListener до Ваших HTML елементів.

2. Додайте можливість підписки не різні івенти (“click”, “mouseover” тощо).

3. Покажіть правильність роботи свого коду запустивши його в головному методі програми.

4. Це завдання виконайте в окремому PR.

Лістинг програми:

LightNode:

```
abstract class LightNode
```

```
{
    public abstract string OuterHTML { get; }
    public abstract string InnerHTML { get; }
     }
```

LightTextNode:

```
class LightTextNode : LightNode
{
    public string Text { get; set; }

    public LightTextNode(string text)
    {
        Text = text;
    }

    public override string OuterHTML => Text;
    public override string InnerHTML => Text;
     }
```

LightElementNode:

```
using System.Text;

enum DisplayType { Block, Inline }
enum CloseType { SelfClosing, Normal }

class LightElementNode : LightNode
{
    public string TagName { get; set; }
    public DisplayType Display { get; set; }
    public CloseType Close { get; set; }

    public List<string> Classes { get; set; } = new List<string>();
    public List<LightNode> Children { get; set; } = new List<LightNode>();


    private Dictionary<string, List<Action>> eventListeners = new Dictionary<string,
List<Action>>();

    public LightElementNode(string tagName, DisplayType display = DisplayType.Block,
CloseType close = CloseType.Normal)
    {
        TagName = tagName;
        Display = display;
        Close = close;
    }

    public void AddChild(LightNode child)
    {
        Children.Add(child);
    }


    public void AddEventListener(string eventName, Action handler)
    {
        if (!eventListeners.ContainsKey(eventName))
        {
            eventListeners[eventName] = new List<Action>();
        }

        eventListeners[eventName].Add(handler);
    }


    public void TriggerEvent(string eventName)
```

```
    {
        if (eventListeners.ContainsKey(eventName))
        {
            foreach (var handler in eventListeners[eventName])
            {
                handler.Invoke();
            }
        }
    }

    public int ChildCount => Children.Count;

    public override string InnerHTML
    {
        get
        {
            StringBuilder sb = new StringBuilder();

            foreach (var child in Children)
            {
                sb.Append(child.OuterHTML);
            }

            return sb.ToString();
        }
    }

    public override string OuterHTML
    {
        get
        {
            string classAttr = Classes.Count > 0
                ? $" class=\"{string.Join(" ", Classes)}\""
                : "";

            if (Close == CloseType.SelfClosing)
            {
                return $"<{TagName}{classAttr}/>";
            }
            else
            {
                return $"<{TagName}{classAttr}>{InnerHTML}</{TagName}>";
            }
        }
    }
     }
```

Program:

```
using System.Text;

class Program
{
    static void Main(string[] args)
    {
        Console.InputEncoding = Encoding.UTF8;
        Console.OutputEncoding = Encoding.UTF8;

        LightElementNode button = new LightElementNode("button", DisplayType.Inline);
        button.AddChild(new LightTextNode("Натисни мене"));


        button.AddEventListener("click", () =>
        {
            Console.WriteLine("Кнопку натиснули!");
```

```
        });


        button.AddEventListener("mouseover", () =>
        {
            Console.WriteLine("Курсор над кнопкою!");
        });

        Console.WriteLine("HTML кнопки:");
        Console.WriteLine(button.OuterHTML);

        Console.WriteLine("\n--- Імітація подій ---");

        button.TriggerEvent("mouseover");
        button.TriggerEvent("click");
    }
  }
```

Результат виконання програми:

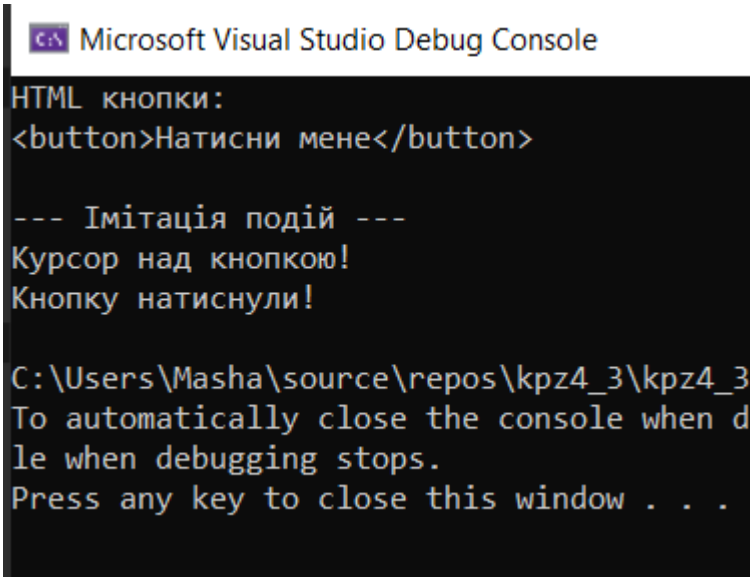

Рис3. Результат виконання програми.

Завдання 4.

1. Додайте до Вашого LightHTML з завдання 5 ЛР №3 новий елемент Image, який за допомогою стратегії в залежності від переданого href буде завантажувати картинку або з файлової системи або з мережі.

2. Покажіть правильність роботи свого коду запустивши його в головному методі програми.

3. Це завдання виконайте в окремому PR.

Лістинг програми:

LightNode:

```
namespace LightHTML
{
    abstract class LightNode
```

```
    {
        public abstract string OuterHTML { get; }
        public abstract string InnerHTML { get; }
    }
 }
```

LightImageNode:

```
using LightHTML;
class LightImageNode : LightNode
{
    public string Href { get; set; }

    private IImageLoadingStrategy strategy;

    public LightImageNode(string href)
    {
        Href = href;

        if (href.StartsWith("http"))
        {
            strategy = new NetworkImageLoadingStrategy();
        }
        else
        {
            strategy = new FileImageLoadingStrategy();
        }
    }

    public void Load()
    {
        strategy.LoadImage(Href);
    }

    public override string OuterHTML => $"<img src=\"{Href}\" />";
    public override string InnerHTML => "";
 }
```

NetworkImageLoadingStrategy:

```
class NetworkImageLoadingStrategy : IImageLoadingStrategy
{
    public void LoadImage(string href)
    {
        Console.WriteLine($"Завантаження з мережі: {href}");
    }
 }
```

FileImageLoadingStrategy:

```
class FileImageLoadingStrategy : IImageLoadingStrategy
{
    public void LoadImage(string href)
    {
        Console.WriteLine($"Завантаження з файлової системи: {href}");
    }
 }
```

IImageLoadingStrategy:

```
interface IImageLoadingStrategy
{
    void LoadImage(string href);
 }
```

Program:

```
class Program
{
    static void Main(string[] args)
    {
        Console.OutputEncoding = System.Text.Encoding.UTF8;

        LightImageNode localImage = new LightImageNode("images/photo.png");

        LightImageNode webImage = new LightImageNode("https://example.com/photo.jpg");

        Console.WriteLine("HTML елемент:");
        Console.WriteLine(localImage.OuterHTML);
        Console.WriteLine(webImage.OuterHTML);

        Console.WriteLine("\nЗавантаження зображень:");

        localImage.Load();
        webImage.Load();
    }
 }
```

Результат виконання програми:

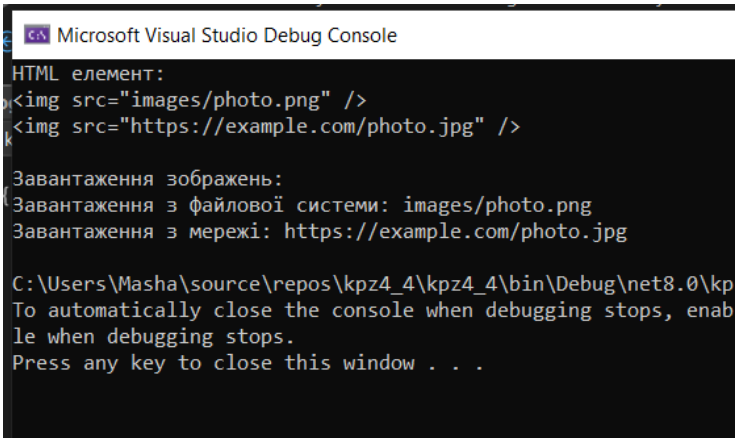

Рис4. Результат виконання програми.

Завдання 5.

1. Уявіть, що ви створюєте програму текстового редактора.

2. Для цього завдання буде достатньо POC-версії: клас TextEditor, який містить в собі поточну версію текстового документу, який представлений спеціальним класо TextDocument. Назви класам можете давати власні.

3. Реалізуйте функцію збереження і скасування, яка дозволить користувачам відмінити зміни, зроблені в документі. Завдання полягає в тому, щоб зберегти стан документа в різні моменти часу та відновити його, коли це необхідно, не розкриваючи внутрішню реалізацію документа чи текстового редактору.

Лістинг програми:

TextEditor:

```
using System.Collections.Generic;

class TextEditor
{
    private TextDocument document;
    private Stack<DocumentMemento> history = new Stack<DocumentMemento>();

    public TextEditor(TextDocument doc)
    {
        document = doc;
    }

    public void Write(string text)
    {
        history.Push(document.Save());
        document.Content += text;
    }

    public void Undo()
    {
        if (history.Count > 0)
        {
            var memento = history.Pop();
            document.Restore(memento);
        }
    }

    public void Show()
    {
        Console.WriteLine(document.Content);
    }
     }
```

DocumentMemento:

```
class DocumentMemento
{
    public string Content { get; }

    public DocumentMemento(string content)
    {
        Content = content;
    }
     }
```

TextDocument:

```
class TextDocument
{
    public string Content { get; set; }

    public TextDocument(string text)
    {
        Content = text;
    }

    public DocumentMemento Save()
    {
        return new DocumentMemento(Content);
    }

    public void Restore(DocumentMemento memento)
    {
        Content = memento.Content;
    }
    }
```

Program:

```
class Program
{
    static void Main()
    {
        TextDocument doc = new TextDocument("Hello");
        TextEditor editor = new TextEditor(doc);

        editor.Show();

        editor.Write(" world");
        editor.Show();

        editor.Write("!!!");
        editor.Show();

        Console.WriteLine("Undo:");

        editor.Undo();
        editor.Show();

        editor.Undo();
        editor.Show();
    }
}
```

Результат виконання програми:

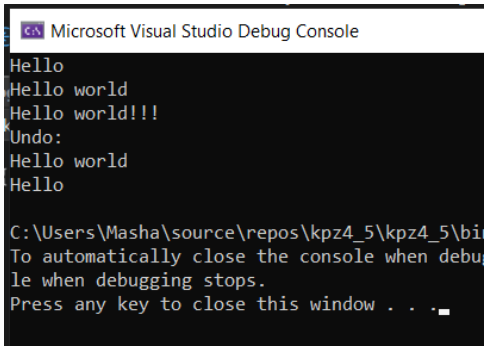

Рис5. Результат виконання програми.

Висновки: навчилася реалізовувати структурні шаблони проєктування Ланцюжок відповідальностей, Посередник, Спостерігач, Стратегія, Мементо